

Simple Wave Function Collapse

Parallélisation OpenMP par API task

Amine Braham & **Antoine Fournaise**

Modèles de programmation parallèle — M1 CHPS
Mai 2026

Plan

Partie 1 — Amine

- Le problème WFC
- Algorithme — vue d'ensemble
- Extraction tuiles & compatibilités
- Entropie, collapse, propagation
- Implémentation série C++

Partie 2 — Antoine

- Stratégie de parallélisation
- Tentative V1 et difficultés
- Architecture finale V2
- Analyse de performance
- Anti-scalabilité & conclusion

Le problème WFC

Wave Function Collapse : technique de génération procédurale qui synthétise une grille de sortie à partir d'un petit échantillon, en imposant que chaque sous-grille $N \times N$ de la sortie existe dans l'échantillon.

Échantillon 5×5

1	0	1	1	1
1	0	1	1	1
0	0	1	1	1
0	1	1	1	1
0	0	0	0	0

Petit motif d'entrée

Approche

- Problème de satisfaction de contraintes (CSP)
- Résolu par propagation locale
- Sortie cohérente *localement* avec l'échantillon
- Applications : textures, niveaux de jeu, art procédural

Algorithme — vue d'ensemble

Cinq étapes principales, répétées jusqu'à convergence ou échec :

#	Étape	Description
1	Extraction	Énumérer les sous-grilles $N \times N$ uniques avec leurs fréquences
2	Compatibilités	Pour chaque paire (t_1, t_2) et offset, vérifier la superposition
3	Entropie	Nombre de tuiles encore possibles par cellule (heuristique MRV)
4	Collapse	Cellule min-entropie $\rightarrow$ choix aléatoire pondéré par fréquence
5	Propagation	BFS de type AC-3 propage les contraintes aux voisins

Le cycle 3-4-5 se répète jusqu'à ce que toutes les cellules soient collapsées (succès) ou qu'une contradiction apparaisse (échec).

Étapes 1 et 2 — Tuiles et compatibilités

Extraction des tuiles

Pour chaque position (r, c) de l'échantillon, on extrait la sous-grille $N \times N$ et on incrémente sa fréquence.

```
for r in 0..H-N:
  for c in 0..W-N:
    t = extract(sample, r, c, N)
    tiles[t].freq += 1
```

Sur l'échantillon 5×5 avec $N=2$: **8 tuiles uniques**.

Table de compatibilité

Précomputation : pour chaque $(t_1, t_2, \text{offset})$, tester si t_1 et t_2 sont compatibles sur leur zone de chevauchement.

```
compatible[t1][oy+OFF][ox+OFF]
= liste des t2 compatibles
  avec t1 à l'offset (ox, oy)
```

Lookup $O(1)$ pendant la propagation.

Étapes 3 à 5 — Résolution

Entropie & Collapse

- **Entropie(c)** = nombre de tuiles encore possibles
- On choisit la cellule de plus faible entropie > 1
- Tirage aléatoire *pondéré par fréquence* parmi les tuiles possibles
- Si entropie = 0 : contradiction → échec

Propagation BFS

- File FIFO initialisée avec la cellule collapsée
- Pour chaque cellule défilée : examiner ses voisins dans $(2N-1)^2$
- Retirer du voisin toutes les tuiles non supportées par la source
- Si le voisin change → remettre en file

Style **AC-3** (Arc Consistency) : chaque modification déclenche une re-vérification des voisins concernés.

Implémentation série en C++17

Choix de design

- Code **templaté** : `WFC<N, T>`
- `N` : taille des tuiles (compile-time)
- `T` : type des valeurs (binaire, multi-valeurs)
- Headers séparés pour série / OMP

Représentation de l'onde

```
using Wave =  
    std::vector<std::vector<bool>>>;  
  
// wave[cell][t] = true ssi  
// la tuile t est encore  
// possible pour cette cellule
```

Entropie = nombre de bits à `true` .

Bénéfice de la généricité : la version multi-valeurs est obtenue gratuitement en instanciant `WFC<2, int>` avec un échantillon à valeurs multiples.

Stratégie de parallélisation

Analyse des phases de l'algorithme pour identifier où paralléliser :

Phase	Parallélisable ?	Granularité
Recherche min-entropie	Oui (réduction)	O(rows·cols) cellules indépendantes
Collapse	Non	RNG global, état partagé
Propagation BFS	Partiellement	O(8) tâches par étape pour N=2

L'énoncé demande explicitement l'API `task` d'OpenMP — choisie pour la propagation où la granularité est variable.

V1 — première tentative

Architecture

- `#pragma omp parallel` créé à chaque étape BFS
- Tâche par voisin
- `#pragma omp critical(wave_update)` englobant toute la mise à jour
- File BFS protégée par `omp_lock_t`

Problèmes constatés

- Le `critical` sérialise **toutes** les tâches
- Aucun parallélisme effectif
- Overhead de création des régions à chaque étape BFS
- $T=32 \rightarrow$ **500× plus lent** que la série

Difficultés rencontrées

1. Critical section globale inutile

La preuve d'absence de race a demandé une analyse fine : pour un `src` fixé, les `dst_j = src + (oy_j, ox_j)` sont **tous distincts** car les offsets sont distincts → pas de verrou nécessaire.

2. BFS : `visited` vs `in_queue`

Premier fix : ajouter `visited[]` pour éviter les doublons → **bug**. Dans WFC une cellule peut nécessiter plusieurs passes si ses voisins changent. La bonne sémantique : `in_queue` réinitialisé au défilement.

3. Overhead de création des régions parallèles

Sur ARM, réveiller des threads dormants coûte **~0.1 à 1 ms par région**. Avec des milliers d'étapes BFS, cet overhead seul dépasse plusieurs secondes — plus que le calcul utile.

V2 — architecture finale

Une seule région parallèle

```
#pragma omp parallel
{
    while (!done) {
        #pragma omp for nowait
        for (cell : ...) // entropie

        #pragma omp single
        {
            collapse(...);
            while (!bfs.empty()) {
                for (j : voisins)
                    #pragma omp task
                    { propage src->dst_j }
                #pragma omp taskwait
            }
        }
    }
}
```

Invariant de sûreté *sans verrou*

- Pour un `src` fixé, les `dst` sont uniques
- Les tâches écrivent dans des cellules **disjointes**
- `wave[src]` est en lecture seule
- Résultats dans `result[j]` (index unique)

Aucun verrou nécessaire sur `wave`. La file BFS est mise à jour *après* le `taskwait`.

Résultats — référence série

Mesures sur nœud Romeo `armgpu` — ARM aarch64, 32 cœurs. Médiane sur 3 exécutions.

Grille	Temps série (binaire)	Temps série (multi)
16×16	~ 0.001 s	0.002 s
32×32	~ 0.003 s	0.004 s
64×64	~ 0.020 s	0.063 s
128×128	~ 0.050 s	0.140 s

Croissance proche du quadratique en nombre de cellules — conforme à la complexité théorique $O(R \cdot C \cdot K)$ où $K = \text{num_tiles}$.

Scalabilité en threads — binaire 128×128

Threads	V1 (région par BFS)	V2 (région unique)	Speedup V2 vs série
T=1	0.054 s	0.045 s	3.1×
T=2	0.985 s	1.218 s	0.12×
T=4	1.714 s	1.690 s	0.08×
T=8	3.767 s	2.852 s	0.05×
T=16	15.506 s	7.486 s	0.019×
T=32	27.068 s	27.956 s	0.005×

Anti-scalabilité : plus on ajoute de threads, plus c'est lent. Confirmé sur les deux architectures.

Pourquoi l'anti-scalabilité ?

Quatre causes identifiées :

(a) Granularité trop fine

8 tâches par étape BFS, chacune ~64 comparaisons booléennes (**quelques ns**).
Overhead OpenMP par tâche : ~ μ s. Ratio défavorable d'un facteur **1000×**.

(b) Dépendance séquentielle

Chaque collapse dépend de l'onde résultant de tous les précédents — la boucle principale est **intrinsèquement séquentielle**. Seul l'intérieur d'une itération peut être parallélisé.

(c) Faux partage sur `result[]`

Le `vector<int>` contigu fait tenir 16 éléments par ligne de cache. 8 threads écrivant simultanément → rebond de ligne de cache, latence multipliée.

(d) Cohérence cache sur `wave`

Toutes les tâches lisent `wave[src]` simultanément. Sur ARM, la diffusion d'une ligne à plusieurs cœurs ajoute de la latence par rapport à un accès local.

Scalabilité en taille — T=16

Binaire

Grille	Temps	µs/cellule	
32×32	0.528 s	516	
64×64	1.307 s	319	
128×128	3.824 s	233	
256×256	21.65 s	330	
512×512	75.10 s	286	

Multi-valeurs

Grille	Temps	µs/cellule	
32×32	0.820 s	801	
64×64	3.454 s	843	
128×128	13.54 s	826	
256×256	45.94 s	701	

Coût marginal stable : ~300 µs/cellule (binaire), ~800 µs/cellule (multi). Plus de tuiles → plus de pression cache.

Pistes pour réellement scaler

1. Bitsets SIMD

Remplacer `vector<bool>` par `uint64_t` (si $K \leq 64$ tuiles) : entropie en 1 instruction `popcount`, AND/OR vectoriels.

Gain attendu : $\times 10$ à $\times 100$

2. Multiple restarts

Lancer T résolutions indépendantes avec graines différentes, garder la première qui converge.

Scalabilité parfaite mais hors cadre du sujet.

3. Coarsening

Regrouper plusieurs cellules sources dans une même tâche. Difficile car les sources du BFS peuvent partager des destinations.

4. GPU / Kokkos

Déplacer entropie et table de compatibilité sur GPU. Calcul d'entropie sur 65k cellules en quelques μs .

Conclusion

Ce qui a été livré

- Implémentation WFC **complète et correcte** en C++17 templaté
- Version série + version parallèle OpenMP utilisant l'**API task explicite**
- Support multi-valeurs *gratuit* via la généricité
- Benchmarks sur cluster HPC avec analyse rigoureuse

Résultat principal

Anti-scalabilité **identifiée, mesurée et expliquée** par quatre mécanismes distincts. WFC à granularité fine est intrinsèquement difficile à paralléliser par tâches.

Comprendre *pourquoi* un algorithme ne scale pas est un résultat académique aussi valable que le faire scaler — il oriente le choix du modèle d'exécution adapté.

Merci de votre attention — Questions ?